

HW2 Report

Task 0

```
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ python3 --version
Python 3.12.3
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ pip show cryptography pytest
Name: cryptography
Version: 49.0.0
Summary: cryptography is a package which provides cryptographic recipes and primitives to
Python developers.
Home-page: https://github.com/pyca/cryptography
Author:
Author-email: The Python Cryptographic Authority and individual contributors
<cryptography-dev@python.org>
License-Expression: Apache-2.0 OR BSD-3-Clause
Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: cffi
Required-by:
---
Name: pytest
Version: 9.1.1
Summary: pytest: simple powerful testing with Python
Home-page: https://docs.pytest.org/en/latest/
Author: Brianna Laughner, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger Krekel,
Others (See AUTHORS), Ronny Pfannschmidt
Author-email:
License-Expression: MIT
Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: iniconfig, packaging, pluggy, pygments
Required-by:
```

Task 1

Result:

```
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ python3 baseline_ctr.py
Original plaintext:{"action":"READ","path":"notes.txt"}
```

Ciphertext:4cd7cb3255249b868bcba21969c85beef56a5110db9f786f9b47a507c71a3ffc774ad500

Normal Receiver behavior

```
{"action":"READ","path":"notes.txt"}
```

XOR difference:

The XOR delta for R and E is 0x17

The XOR delta for E and D is 0x1

The XOR delta for A and I is 0x8

The XOR delta for D and T is 0x10

Modified receiver behavior

```
{"action":"EDIT","path":"notes.txt"}
```

Replay attack

First process:

```
{"action":"READ","path":"notes.txt"}
```

Second process:

```
{"action":"READ","path":"notes.txt"}
```

For my python program I first created an encryption function that turns plaintext into ciphertext using AES-CTR encryption, A function that decrypts ciphertext into plaintext, a receiver function that displays ciphertext into plaintext, and a relay function that changes the action in the command using XOR on ciphertext. The design shows that even without the key and with just encryption itself it's vulnerable to a bit flipping attack.

In my bit flipping attack I changed the action READ into the action EDIT by calculating XOR differences between READ and EDIT and then applied these differences to the ciphertext bytes. I was then able to feed this new command into the receiver which after decryption displayed the new EDIT action instead of the READ. This was all done without being able to see the secret key showing how vulnerable encryption by itself can be without other layers. Furthermore, I was able to replay the original ciphertext two times and the command was processed.

The root cause of the issue is the integrity vulnerability with AES-CTR. This is because AES-CTR uses XOR to convert between plaintext and ciphertext. This means that edits to the ciphertext can create predictable outcomes to the plaintext. This means it is vulnerable to bit flipping attacks that change the ciphertext without having to know the key. Lastly, without relay protection the ciphertext can be replayed and reprocessed.

Task 2

For the handshake python program the design uses a Diffie-Hellman key establishment so that a secret between the gateway and the node can be created without having to directly send the secret. Each side of the handshake generates a new DH key as well as 16 byte nonces for every new session. This makes it so that each handshake is unique and provides replay resistance. It also provides forward secrecy because even if other long term keys such as the RSA keys are revealed, new DH values will change with each session. Then each side contains a digital signature so that integrity can be guaranteed between sender and receiver. This is done with a RSA key to authenticate each side. The transcript is also built with both sides' identities, roles, nonces and Diffie Hellman values so that basic man in the middle attacks are prevented. Each value is authenticated and protected by signature verification meaning that if anything is changed it will become invalid. Reflection attacks are also much harder to do because each side of the handshake has their own role and cannot be reused in the opposite direction. The design also uses key separation by creating a separate encryption and MAC key for each direction. If the RSA signatures were removed, the diffie hellman secret could no longer be guaranteed safe. DH values could be open to man in the middle attack and a third party secret could be established with each side.

Task 3

Output example of `secure_record.py`:

```
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ python3 secure_record.py
```

Record:

[illegible]

Recovered plaintext:

```
{"action": "READ", "path": "notes.txt"}
```

Task 4

Test_records.py result:

```
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ python -m pytest tests/test_records.py
```

$$-V$$

```
===== test session starts
```

=====

platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 --

```
/home/ubuntu/csce465-agentsec/.venv/bin/python
```

cachedir: .pytest_cache

```
rootdir: /home/ubuntu/csce465-agentsec/hw2
```

collected 4 items

```
tests/test_records.py::test_modified_ciphertext PASSED [ 25%]
tests/test_records.py::test_modified_header PASSED [ 50%]
tests/test_records.py::test_replayed_record PASSED [ 75%]
tests/test_records.py::test_reflected_record PASSED [100%]
```

===== 4 passed in 0.27s

=====

Test_handshake.py result:

(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2\$ python -m pytest

tests/test_handshake.py -v

===== test session starts

=====

platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 --

/home/ubuntu/csce465-agentsec/.venv/bin/python

cachedir: .pytest_cache

rootdir: /home/ubuntu/csce465-agentsec/hw2

collected 4 items

```
tests/test_handshake.py::test_valid_handshake PASSED [ 25%]
tests/test_handshake.py::test_incorrect_rsa_public_key PASSED [ 50%]
tests/test_handshake.py::test_invalid_transcript PASSED [ 75%]
tests/test_handshake.py::test_reflected_handshake_message PASSED [100%]
```

===== 4 passed in 3.25s

=====

The attacker model for my design assumes that an attacker can see, modify, replay and reflect records sent between the gateway and node. The record layer attempts to provide confidentiality and integrity of the message between the two sides. It prevents a message from being replayed by tracking sequence and prevents reflection attacks by creating unique roles for each side. However, it does not check the message itself to be safe or a command is allowed. Instead it simply makes sure that the original message is not tampered with. This can create issues when the original message itself is not a safe command.

CTR requires the use of unique IVs because its behaviour is predictable. This is because it always creates a keystream that is XORed with the plaintext to get the ciphertext. This keystream is made from the encryption key and IV. Therefore when you're using the same encryption key the IV has to change otherwise the relationship between plaintext can be solved.

The IV is also built using the sequence number which is constantly changing as messages are being sent back and forth. As a result IV's are constantly changing as well.

Verifying before decrypting matters because if records have been tampered with it should not be allowed through in case the altered plaintext is dangerous. For the design it uses HMAC to authenticate the header, IV and cipher text string to make sure that it has not changed. Verifying before decryption makes sure that corrupted plaintext is not released after decryption. This is mainly important when the original plaintext is a dangerous command or tool call.

Independent encryption and MAC keys for each side are implemented to provide key separation. Each key instead of being used for every operation it is instead used for specific cases. For example the encryption key is used to guarantee confidentiality while the MAC keys are used for integrity of the record. Since each side has their own independent keys, the keys are also unique to their direction. When comparing this construction to an AEAD mode such as AES-GCM, it takes a lot more manual implementation. For example you first have to use AES-CTR to create your ciphertext. You then have to use HMAC on your created header, IV and ciphertext to get your tag. When it comes to AES-GCM does encryption and authentication in one operation to create the ciphertext and tag.

A valid protected record may still contain a dangerous authorized tool call because the design does not actually check if the record being sent over is safe or not. It simply guarantees the confidentiality and integrity of the message. An example where a dangerous tool call could be sent through would be if the original message was a dangerous tool call. The record creation would be designed to keep this tool call protected and sent over to the other side to make sure it was not tampered with. So although the command was technically a protected record, it was simply always malicious and didn't require tampering to send a dangerous tool call.